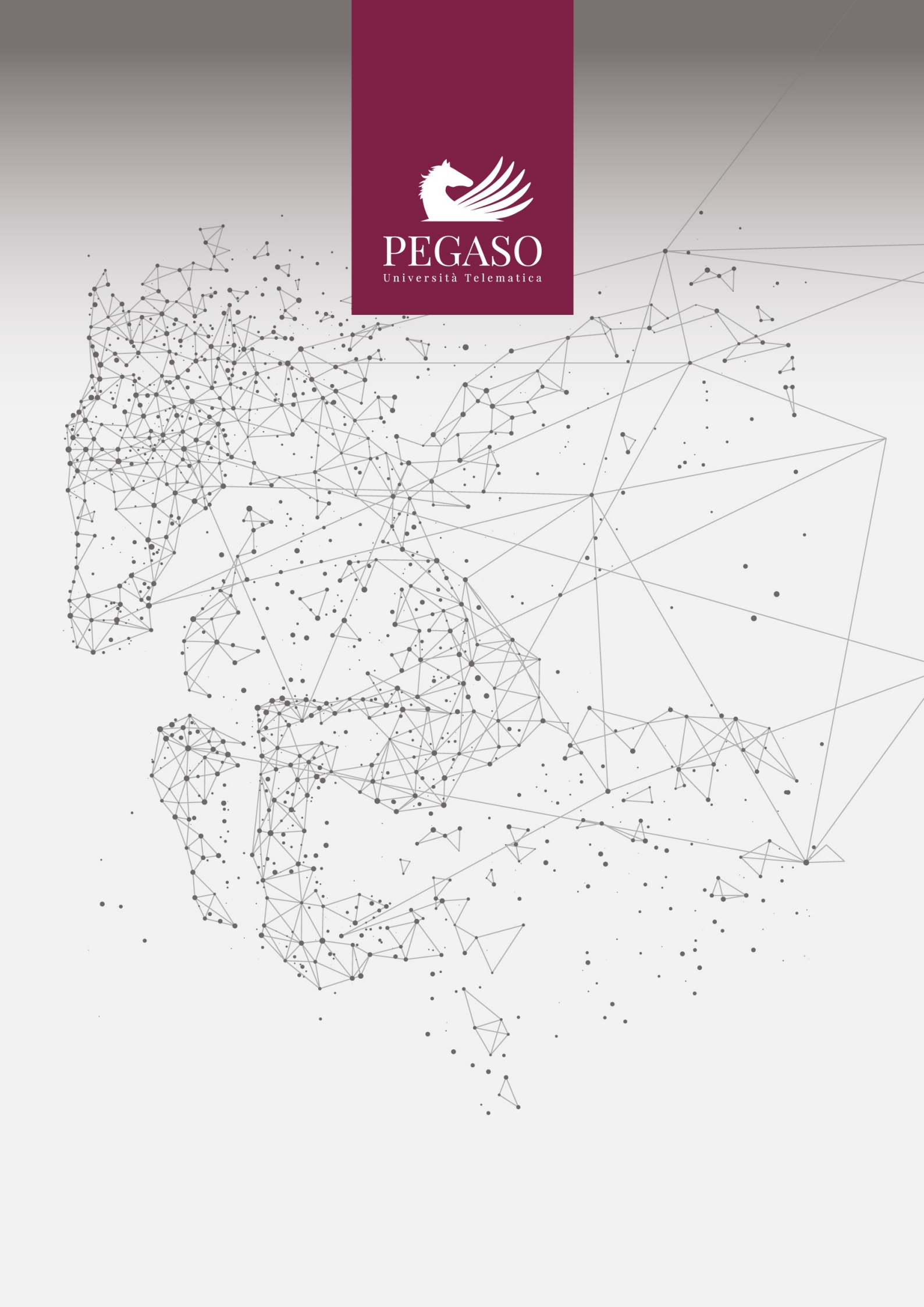

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. COME USARE I DIAGRAMMI UML....3
2. ALTRI DIAGRAMMI UML 7
3. CONSIDERAZIONI GENERALI SU UML....13
BIBLIOGRAFIA 19

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Come usare i diagrammi UML

I diagrammi UML offrono viste diverse del sistema a seconda della prospettiva con cui si vuole modellare il sistema. Ogni diagramma ha quindi un ruolo diverso e il suo uso dipende dallo scopo di modellazione e dalla fase di utilizCome usare i diagrammi delle classi

a. Come usare i diagrammi delle classi

Il diagramma delle classi offre una descrizione statica (ovvero strutturale) dei delle classi che compongono il sistema e delle relazioni tra di esse. Nelle versioni più dettagliate mostrano anche gli attibuti (stato) e le operazioni.

I diagrammi delle classi sono la spina dorsale di UML e sono largamente usati. La ricchezza di elementi e notazioni che mette a disposizione li rende uno strumento molto flessibile che può essere usato da figure diverse e in divese situazioni di utilizzo.

Gli elementi di base dei diagrammi sono usati da tutti, quelli avanzati un po' meno ma senza dubbio sono molto usati perché risultano utili sia in fase di discussione del problema, analisi dei requisiti e di progettazione vera e propria.

Il punto di forza del diagramma delle classe può però rappresentare al tempo stesso un problema. La ricchezza di elementi e notazioni può infatti mettere l'utente nella situazione di essere travolto dalla complessità descrittiva che ne può derivare senza ricavare alcuna utilità dal diagramma.

Ecco qualche consiglio per il loro uso.

Manterenere la notazione più semplice possibile cercare di non usare tutte le notazioni disponibili. Cominciare con i concetti più semplici: classi, associazioni, attributi, generalizzazione e vincoli. È meglio rimandare l'uso dei concetti più avanzati (ad esempio interfacce astratte) al momento in cui questi si riveleranno davvero necessari.

I diagrammi delle classi disegnati da una prospettiva concettuale (per la descrizione dei concetti nel dominio studiato) sono molto utili per esplorare il vocabolario di un determinate dominio applicativo (ad esempio in ambito di transizioni finanziarie, redazione di bilanci patrimoniali) o di un'organizzazione (ad esempio in ambito business). Per farlo al meglio, però, bisogna sforzarsi di lasciare gli aspetti software fuori dal discorso e manterenere la notazione più semplice possibile.

Si raccomanda di non disegnare un diagramma per ogni cosa; al contrario concentratevi sugli aspetti più importanti. È meglio avere pochi diagramma aggiornati e usati spesso piuttosto che un mucchio di modelli obsoleti, presto dimenticati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il pericolo maggiore è focalizzarsi esclusivamente sulla struttura del sistema, trascurando il suo comportamento dinamico. Quando disegnate diagrammi delle classi per comprendere il funzionamento di un software, uniteli sempre a qualche tecnica di documentazione sul comportamento. Se vi accorgete che passate spesso da una tecnica all'altra, significa che state lavorando bene.

b. Come usare i diagrammi di casi d'uso

I casi d'uso aiutano a rappresentare come il sistema può essere usato nelle diverse situazioni e quindi cosa il sistema deve fare o ci aspettiamo che faccia. Essi quindi forniscono una visione del sistema dall'esterno. Per questo motivo, i casi d'uso sono uno strumento prezioso per comprendere i requisiti funzionali di un sistema. Una loro prima analisi dovrebbe essere svolta nelle fasi iniziali del progetto per rappresentare i requisiti; versioni più dettagliate dei vari casi d'uso potranno essere sviluppate appena prima dell'implementazione per rappresentare il funzionamento da realizzare. Proprio perché i casi d'uso rappresentano una visione del sistema dall'esterno e descrivono il sistema dalla prospettiva dell'utente, non c'è da aspettarsi alcuna correlazione tra i casi d'uso e le classi che compongono il sistema software.

La maggiore significatività è data dalla descrizione dei casi d'uso piuttosto che dai diagrammi di casi d'uso. Si consiglia in primis porre maggior attenzione ai primi, ovvero di concentrarsi principalmente sulle loro descrizioni testuali. Nonostante il fatto che UML non ne parli affatto, l'utilità di questa tecnica è racchiusa tutta nel testo che esprime il contenuto dei casi d'suo.

Un grande pericolo dei casi d'uso è renderli troppo complicati, fino al punto di trovarsi impastoiati nei dettagli: viceversa, svilupparli troppo poco è un rischio minore. Nella maggior parte delle situazioni, un paio di paginette per caso d'uso vanno più che bene. Nei casi d'uso è meglio avere poche informazioni che troppe. Se scrivete troppo poco, almeno avrete un documento breve e leggibile che si può usare come base di partenza. Se avrete scritto troppo, sarà difficile che qualcuno legga e comprenda il vostro lavoro rendendo di fatto lo sforzo fatto quasi inutile.

In generale si consiglia quindi di descrivere i casi d'uso in modo conciso per garantire facilità di lettura e passaggio di conoscenze. La quantità di dettagli inclusi nella descrizione di un caso d'uso dipenda anche dalla fase in cui sono utilizzati. Spesso all'inizio del progetto è necessario definire con precisione solo pochi casi davvero critici; gli altri possono essere sviluppati al momento dell'implementazione. Non c'è neanche bisogno di mettere su carta tutto quanto; la comunicazione verbale può bastare, soprattutto se si adotta un processo iterative in cui è più facile scrivere codici in tempi brevi che produrre documenti cartacei.

c. Come usare i diagrammi di sequenza

Il diagramma di sequenza è il più popolare tra i diagramma di interazione (a sua volta appartenenti ai diagrammi di comportamento) e descrive la collaborazione di un gruppo di oggetti che devono

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

implementare collettivamente un certo comportamento. A tal fine, il diagramma mostra la sequenza temporale dei messaggi che gli oggetti coinvolti si scambiano al fine di realizzare tale comportamento.

I diagrammi di sequenza sono da usarsi ogni volta che è necessario rappresentare il comportamento di un insieme di oggetti all'interno di un singolo caso d'uso.

Un diagramma di sequenza documenta tipicamente il comportamento di un singolo scenario. Infatti si accompagna per questo spesso ad un diagramma di caso d'uso descrivendo le interazioni del caso d'uso in termini di chiamate e messaggi scambiati tra un set di oggetti coinvolti. Inoltre può anche completare un diagramma della classe mostrandone il comportamento dinamico di tali classi tramite invocazioni di un'operazione e/o invio di messaggi. Non si vedono le associazioni tra oggetti. Non sono precisi nel rappresentare il comportamento dei singoli oggetti, in tal caso si raccomanda l'uso dei diagrammi di macchina a stati o il diagramma delle attività.

Quando si disegnano diagrammi di sequenza, occorre prestare attenzione a non aggiungere troppi dettagli per non ingombrare il diagramma rendendolo difficile da leggere (ad esempio mostrando i messaggi di risposta quando non strettamente necessari). Inoltre si raccomanda di tenere aggiornati i diagrammi in linea con le interfacce e l'architettura reali. Evitando questi errori potete garantire la qualità visiva e comunicativa del diagramma suo vero punto di forza come tutti i diagrammi UML.

d. Come usare i diagrammi di macchina a stati

I diagrammi di macchina a stati descrivono efficacemente il comportamento di un oggetto in più casi d'uso, mentre non funzionano bene quando bisogna rappresentare la collaborazione tra più oggetti. Di conseguenza conviene combinarli con altre tecniche, come i diagrammi di sequenza per descrivere il comportamento di molti oggetti in un singolo caso d'uso, oppure i diagrammi di attività per mostrare la sequenza generale delle attività svolte più oggetti in casi d'uso differenti.

In generale, le machine a stati sono adatte a modellare elementi che reagiscono a degli stimoli, prevalentemente esterni, ma non quelli che esibiscono behaviour proattivo, ovvero che sono capaci di attività autonoma.

Se si usano i diagrammi di macchina a stati, non importa disegnarne uno per ogni classe del sistema. Sebbene questo venga fatto da qualcuno, è quasi sempre una perdita di tempo. E' bene rappresentare con un diagramma solamente le classi che hanno una logica interna interessante e complessa, per aiutarvi a comprendere il loro funzionamento. Molte volte gli oggetti di controllo e i component delle interface grafiche esibiscono dei comportamenti che ben si prestano a essere rappresentatii con diagramma di stato.

Non tutti trovano naturale esprimersi con i diagrammi di stato; per questo motivo dovreste controllare l'uso che ne fanno i componenti del gruppo. Potrebbe anche darsi che questi diagrammi non si

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

adattino al vostro stile di lavoro. Questo non è un grosso problema; come al solito dovete usare sempre l'insieme di tecniche che meglio si adattano meglio a voi.

e. Come usare i diagrammi di attività

Il diagramma di attività permette di illustrare il flusso delle lavoro (intesa come una sequenza di azioni) di un sistema (ovvero workflow o logica procedurale). Possono essere utilizzati per rappresentare i processi aziendali al fine di descrivere il flusso di lavoro di un processo aziendale (ad esempio il flusso e il controllo delle decisioni o della operatività aziendale. Possono altresì servire per modellare il flusso di azioni in un caso d'uso in fase di analisi, per modellare un'operazione di una classe o un algoritmo in fase di progettazione.

La grande forza dei diagrammi di attività sta nel fatto che supportano e incoraggiano l'espressione parallela del comportamento. Questo fa di loro un grande strumento per la modellazione di processi e workflow.

I diagrammi di attività si possono usare per rappresentare dei semplici flowchart: i *fork* e i *join* possono aiutare a descrivere gli algoritmi paralleli in ambito di programmazione concorrente. Tuttavia, non risulta un grande uso in tale direzione probabilmente perché ai fini della regolazione dell'accesso condiviso ai dati (vero aspetto di complessità in programmazione concorrente) i diagrammi di attività non sono particolamente utili.

Nonostante la notazione sia simile, essendo entrambi grafi orientate, bisogna ricordare che il diagramma di macchina a stati e di attività sono totalmente diversi in quanto i nodi delle state machine descrivono stati e le azioni sono sulla transizioni, mentre nel caso delle activity i nodi rappresentano le azioni e gli archi il flusso dei dati e del controllo. Se il focus è mettere in ordine un insieme di azioni da fare o fatte da una o più entità, allora conviene usare il diagramma di attività. Se il focus è mostrare l'evoluzione di un'entità in risposta ad eventi allora conviene usare diagramma di macchina a stati.

.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Altri diagrammi UML

I diagrammi UML si differenziano per il particolare aspetto considerato nella modellazione ovvero per il tipo di vista che offrono del sistema. I diagrammi sono in tutto 13 e sono divisi tra diagrammi di struttura e di comportamento:

- Modelli di struttura: descrivono la struttura di un sistema fornendo una fotografia dei vari elementi (componenti, classi, oggetti) ad un dato istante, rappresentando quindi una vista statica del sistema.
- Modelli di comportamento: catturano gli aspetti dinamici del sistema ovvero come gli elementi del sistema evolvono nel tempo.

Di seguito una panoramica dei seguenti diagrammi UML in aggiunta a quelli già esaminati: diagramma degli oggetti, diagramma di package, diagramma di deployment, diagramma di temporizzazione.

1. Diagramma degli oggetti

Come diagramma di struttura, il diagramma degli oggetti mostra una fotografia in un dato momento delle istanze di oggetti che compongono il sistema software insieme alle relazioni tra gli oggetti mostrati (istanze delle associazioni). Dal momento che mostra istanze piuttosto che classi, è anche detto diagramma delle istanze.

La sintassi visuale prevede che le istanze degli oggetti rappresentate da un rettangolo come le classi, con la differenza che il nome dell'oggetto e della classe sono sottolineati per indicare che si tratta di un'istanza. Inoltre possono essere indicati uno o più attributi se questo risultasse significativo ai fini della leggibilità e comprensione del diagramma. Il formato per indicare il nome dell'oggetto è nome-istanza:nome-classe (può comparire o solo nome-istanza oppure solo :nome-classe). Per indicare eventualmente gli attributi il formato è nome-attributo = «valore» con l'elenco degli attributi non necessariamente esaustivo.

È importante precisare che gli elementi di un diagramma ad oggetti non sono vere e proprie istanze bensì **specifiche di istanze**: si possono omettere attributi obbligatori e indicare solo le informazioni sullo stato delle istanze che si ritengono interessanti, quindi non è possibile da quanto presente nel diagramma ricostruire l'intero oggetto. Inoltre è possibile mostrare istanze di classi astratte (che in realtà non sono istanziabili).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si può usare un diagramma degli oggetti per rappresentare una configurazione esemplificativa di oggetti in un dato istante. Questo è molto utile quando le possibili connessioni tra gli oggetti sono complicate e/o si vuole catturare le informazioni relative sulle relazioni di particolari istanze di oggetti.

In Fig. 1 è mostrato a sinistra un insieme di classi, mentre a destra un insieme di oggetti che sono istanze di quelle classi. Il diagramma delle classi si riferisce alla struttura di composizione di un gruppo. Le classi Persona e Organizzazione sono una specializzazione della classe Gruppo, esse ereditano l'attributo "luogo" e sono anche automaticamente istanze della classe Gruppo. Quindi l'associazione indicata può esserci anche tra istanze di oggetti Organizzazione cosi come tra oggetti Persona e Organizzazione.

Il diagramma degli oggetti mostra istanze esemplificative di gruppi in cui l'organizzazione di Milano è padre di quella di Roma e Genova e per l'organizzazione di Roma è padre di due istanze degli oggetti Persona.

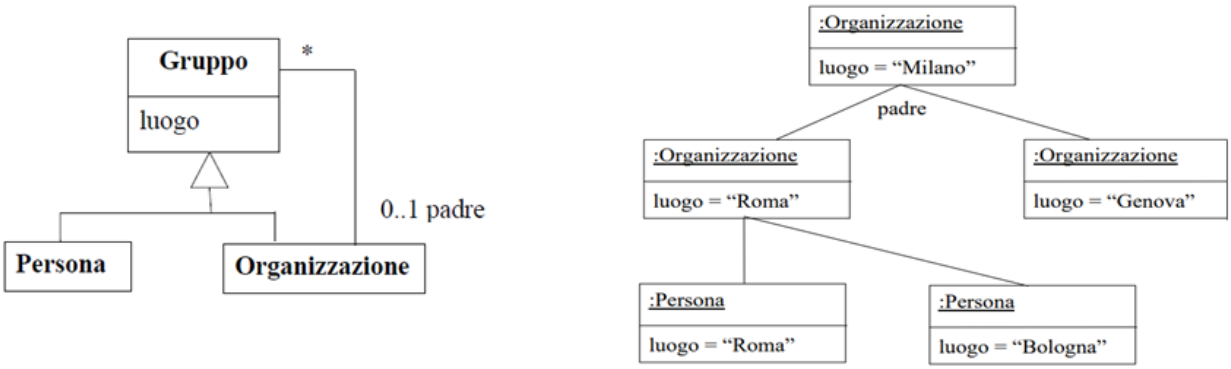


Fig. 1: Esempi di diagramma delle classe (sinistra) e degli oggetti (destra)

2. Diagramma di package

A livello implementativo, le classi di un sistema software realistico nell'ambito industriale possono arrivare ad essere anche centinaia e sono suddivise in librerie, mediante meccanismi di namespace (C++ e C#) oppure package (Java). Analoghi meccanismi di raggruppamento per le classi sono utili anche a livello di modellazione (riferito al momento della compilazione), al fine di dominare la complessità strutturale di un sistema software. UML prevede a tale scopo i diagramma di package.

Un package è un costrutto che permette di prendere un numero arbitrario di elementi UML e raggrupparli insieme in unità di livello più alto. In genere i package si usano per riunire classi. In un modello, ogni classe fa parte di un solo package. Questo può essere a sua volta membro di un altro package e così via andando a formare in definitiva una gerarchia nella quale i package di livello più alto sono scomponibili in sotto-package che a loro volta ne contengono altri e cosi via fino a che gli elementi della gerarchia non si riducono a singole classi. Un package può contenere sia sotto-package che classi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La notazione grafica del package è un rettangolo dotato di linguetta in alto a sinistra

I diagrammi di package sono utili nel progetto di sistemi di grande scala, per documentare le dipendenze tra i componenti più importanti e tenere sotto controllo la complessità strutturale del codice. Fra due package c'è dipendenza se c'è dipendenza fra (almeno) una classe di un package e una classe (necessariamente pubblica) di un altro package. In Fig. 2 si nota una dipendenza tra il package di presentazione e quello di dominio segno della dipendenza di almeno una classi di presentazione da un oggetto del dominio. In altre parole, le dipendenze tra i package riassumono quelle tra gli elementi contenuti.

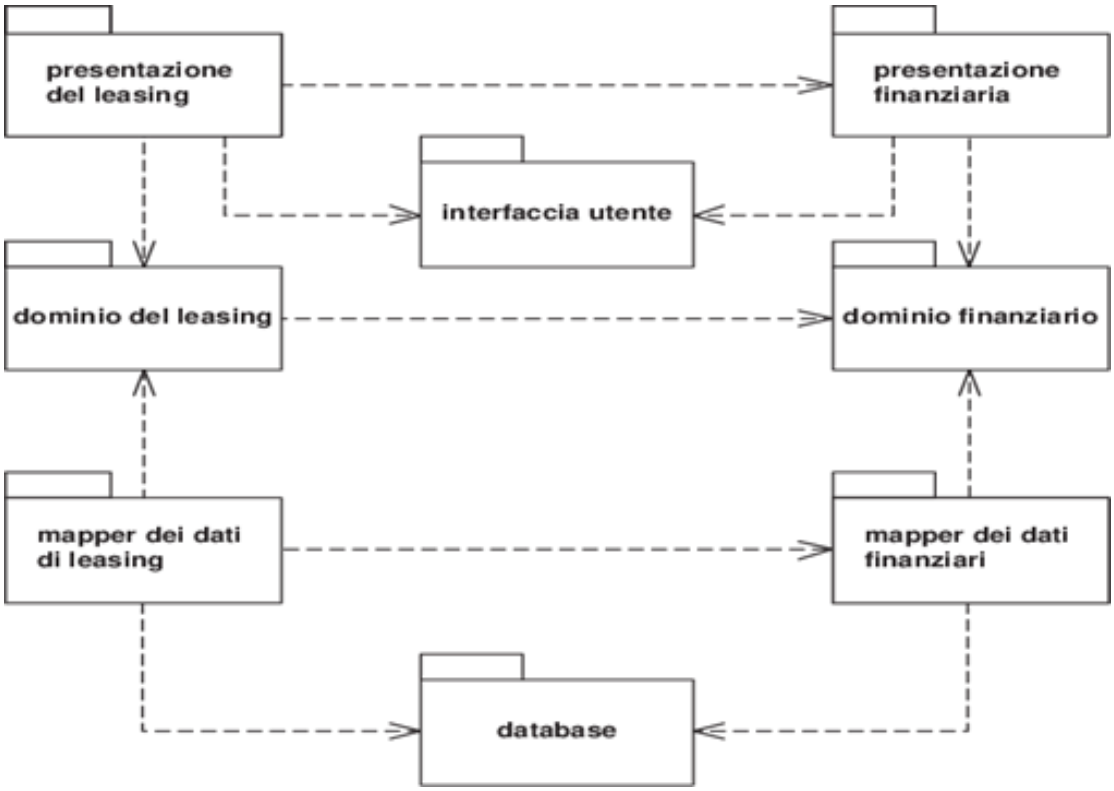


Fig. 2: Diagramma dei package per un'applicazione industriale

3. Diagramma di deployment

Spesso infatti per il dispiegamento in esercizio (deployment) del sistema software si usano più nodi, ovvero svariate macchine (server) o più ambienti di esecuzione (sistemi operativi, ambienti di virtualizzazione, piattaforme). Per cui è importante avere un diagramma che documenti la distribuzione fisica in un sistema, mostrando su quali nodi della rete fisica sono in esecuzione i vari pezzi software, le loro caratteristiche e gli elaborati in esecuzione su tali nodi. Gli elaborati possono essere file eseguibili, file di dati o di configurazione, documenti HTML. Inoltre è importante indicare anche come i nodi sono collegati

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

per rappresentare i flussi o canali di comunicazione fra di essi caratterizzati ad esempio dai protocolli usati per la trasmissione dati o comunicazioni. Per trasmettere l'idea che gli elementi di un diagramma di deployment rappresentano oggetti fisici, la notazione grafica usata per i node è un parallelepipedo che può anche essere sostituito da icone più suggestive, come immagini di computer o cilindri per rappresentare database. In Fig. 3 si può vedere un esempio di diagramma di deployment per un sito web: si possono notare i nodi client collegati al server web il quale a sua volta è collegato con nodi server di back-end dove componenti applicative aggiuntive sono in esecuzione. L'elaborato può anche essere indicato con una rappresentazione grafica a parte per indicare informazioni aggiuntive (ad esempio il vendor).

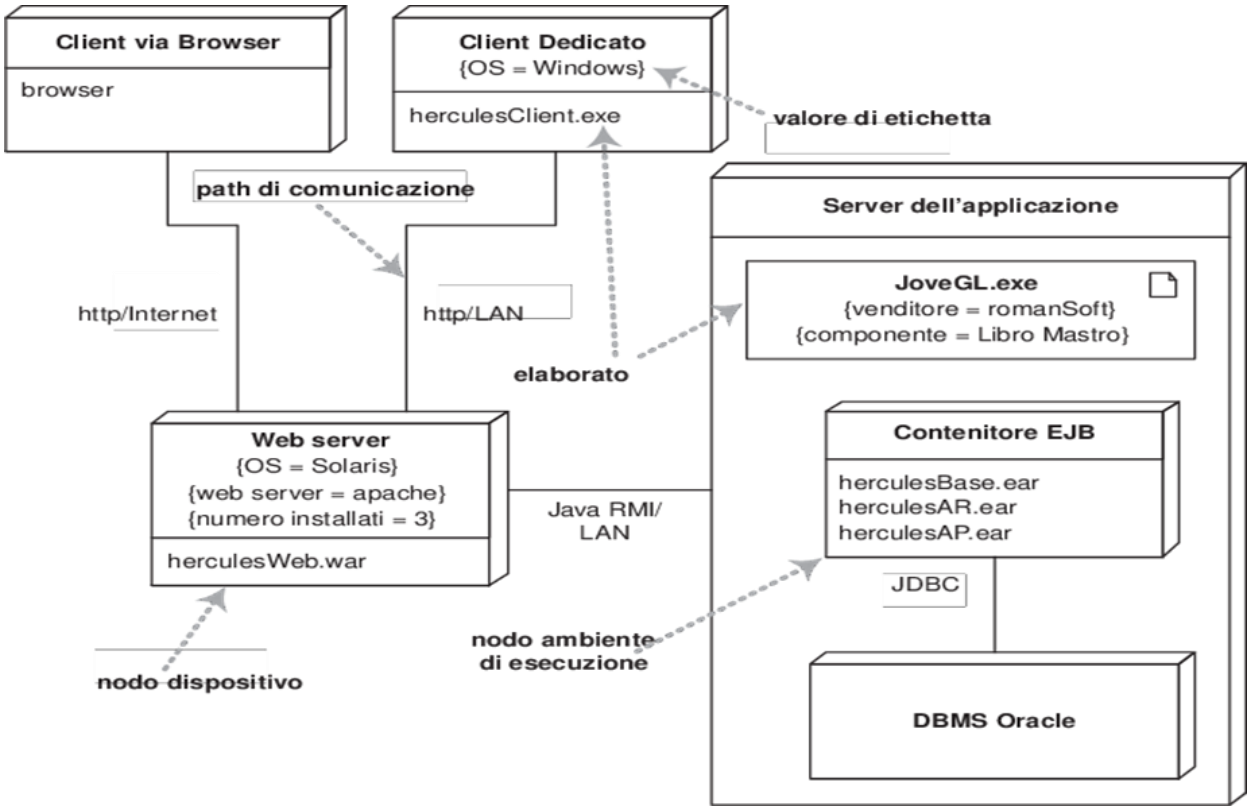


Fig. 3: Diagramma di deploymnent per il software di un sito web.

4. Diagramma di temporizzazione

I diagrammi di temporizzazione sono un'altra forma di diagrammi di interazione, dedicata specificatamente all'espressione di vincoli temporali. Si possono applicare ad un oggetto singolo o anche ad un intero gruppo di oggetti.

In particolare, i diagrammi di temporizzazione descrivono le interazioni mettendo in risalto gli aspetti temporali e di cambiamento di stato. Per fare questo si rappresenta ciascun oggetto (e quindi la

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

relativa lifeline) in un riquadro separato riportando sull'asse verticale i suoi possibili stati. Sull'asse orizzontale, invece, comune a tutto il diagramma, si riporta il tempo, indicando gli istanti interessanti, in primis quelli in cui avvengono i cambi di stato. Per ciascun oggetto è a questo punto possibile indicare mediante una spezzata in quale stato si trova, alzando o abbassando la linea corrispondente. E' anche possibile indicare dei vincoli specifici, ad esempio sulle durate minime o massime di intervalli di tempo sullo stato di un oggetto, e degli eventi se questi sono significativi per il cambio di stato.

I diagrammi di temporizzazione sono particolarmente familiari agli ingegneri che si occupano di hardware. Si possono applicare a un oggetto singolo o, in modo più utile, a un intero gruppo di oggetti.

In Fig. 4 è rappresentato un esempio di diagramma di temporizzazione che si riferisce ad una pompa e una resistenza elettrica di una caffettiera e ai vincoli temporali che esistono per far farli funzionare insieme utilmente: devono passare almeno 10 secondi tra l'accensione della pompa e quella della resistenza, quando il serbatoio dell'acqua si svuota, la pompa si spegne e a da quel momento la resistenza non può rimanere accesa per più di 15 minuti.

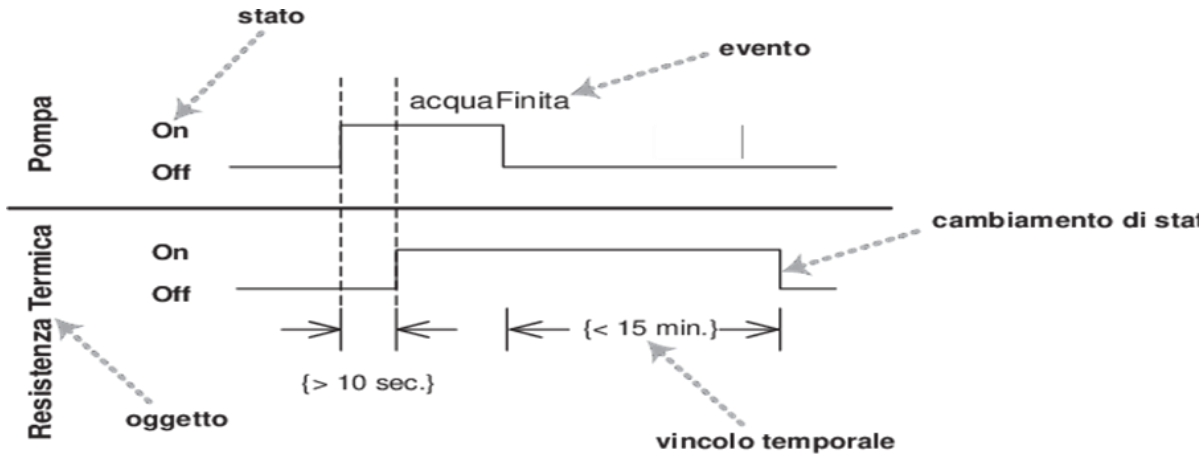


Fig. 4: Diagramma di temporizzazione in cui gli stati sono rappresentati da linee

Esiste anche una rappresentazione alternativa per i diagrammi di temporizzazione che è mostrata in Fig. 5.

Quest'ultimo tipo di rappresentazione può essere utile specialmente in presenza di stati multipli che nel primo tipo di rappresentazione vanno ad arricchire le opzioni rappresentati sull'asse verticale mentre nel secondo tipo non vi è alcun spazio occupato in più in quanto apparirebbero come nomi diversi all'interno delle apposite aree.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

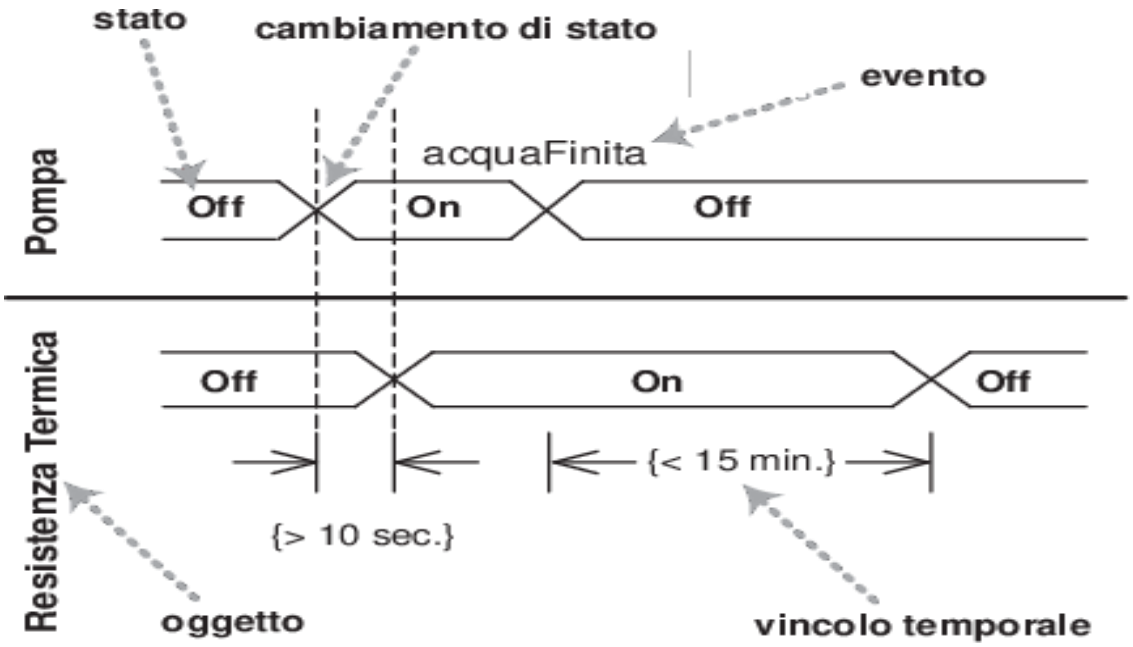


Fig. 5: Diagramma di temporizzazione in cui gli stati sono rappresentati come aree

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Considerazioni generali su UML

a. Motivi di successo di UML

UML rappresenta un linguaggio di modellazione molto affermato nella comunità di chi produce e sviluppa software. Vi sono molte ragioni per cui UML ha prevalso come linguaggio di modellazione.

Prima di tutto si tratta di una notazione visuale e per questo UML fornisce un valido aiuto alla comprensione e alla comunicazione. Un buon diagramma spesso aiuta a trasmettere le idee che sono alla base di un progetto. La notazione visuale permette una più facile comprensione anche per utenti non esperti, ma anche potente per essere usata da utenti esperti ed esprimere tutta la complessità del sistema quando serve. Anzi, più propriamente, si tratta di una famiglia di notazioni diverse, ciascuna perfezionata per esprimere un particolare aspetto del sistema, ma integrate in modo da modellare le sfaccettature di un'entità in modo integrato. Tramite UML si può produrre uno o più modelli, cioè collezioni di diagrammi che rappresentano vari aspetti delle entità da modellare, al livello di astrazione richiesto dall'attività all'interno della quale si svolge la l'analisi e la progettazione di un sistema software.

Inoltre non si tratta di notazioni decise arbitrariamente, ma della evoluzione di linguaggi già in uso, che avevano già incontrato il favore degli utenti ed erano familiari a molti. La diffusione di UML si deve proprio a questo, al fatto che fosse già in uso e di fatto standardizzato all'interno della comunità che si occupa di sviluppo orientato agli oggetti. In tale ambito UML è diventata la notazione grafica dominante.

Infine, grazie alla sua flessibilità e efficacia di rappresentazione l'uso di UML si esteso anche in altri ambiti.

È vero che UML è stato creato per modellare comunità di oggetti. Se inizialmente s'interpretava oggetto in senso strettamente tecnico (oggetti software) e ci si restringeva quindi alla modellazione di sistemi software (come è stato mostrato in queste lezioni) negli anni successivi ci si è resi conto che UML era in grado di modellare altrettanto bene comunità di oggetti in senso lato, ad esempio organizzazioni o parti del mondo reale.

Pertanto il suo uso è stato esteso anche alla modellazione del dominio applicativo (di applicazione), del business e delle imprese, contribuendo al successo ed alla diffusione di UML. Per esempio, si può usare UML per provare a capire e organizzare concettualmente i molti significati del termine *beni patrimoniali* con un gruppo di contabili. In questo caso non si parla più di elementi software, ma ci concentra invece sulla costruzione di un vocabolario comune utile per discutere un particolare dominio. Oppure si può usare

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

UML per rappresentare il flusso di lavoro di un processo aziendale (ad esempio il flusso e il controllo delle decisioni o della operatività aziendale di un certo reparto oppure fra reparti).

Oltre all'utilizzo in domini di oggetti differenti, UML è diventato popolare anche tra chi adotta tecniche non object-oriented in quanto fornisce comunque uno strumento utile alla comunicazione e alla comprensione delle idee alla base di un progetto.

b. Consigli generali di utilizzo di UML

La regola generale nell'utilizzo di UML è sfruttare la molteplicità dei diagrammi offerti per rappresentare vari aspetti delle entità da modellare e di modulare la rappresentazione dei dettagli a seconda della situazione di utilizzo. Anzi si consiglia di cercare di non includere eccessivi dettagli mai e comunque di mostrarne in base alle effettive necessità. UML permette di suddividere le informazioni relative ad uno stesso aspetto del modello su più diagrammi, graduando la precisione di essi ed omettendo, ove necessario, parte dei dettagli.

Quindi in ogni singolo diagramma non è richiesto di inserire tutti gli elementi del modello che vi possono comparire, né, se si sceglie di inserirli, di indicarne tutti i dettagli. Inoltre non è richiesto di modellare ogni parte del sistema oppure di farlo con lo stesso livello di dettaglio per tutte le parti.

Ad esempio, vi possono essere più Class Diagram ciascuno focalizzato su una parte del sistema che realizza alcune funzionalità, per cui le classi non rilevanti per la parte del sistema che si sta descrivendo sono omesse, oppure compaiono con solo il nome, quando ad esempio sono utilizzate per indicare una relazione e saranno descritte pienamente in altri Class Diagram.

È anche possibile rappresentare alcuni attributi o operazioni di una particolare classe in un class diagram ed altri in un altro, ad esempio, evidenziando in ciascun class diagram solo le funzionalità rilevanti per le altre classi. Analogamente avviene per altri tipi di elementi in altri diagrammi.

Inoltre non è necessario esprimere tutti i dettagli necessari in un'unica sessione di lavoro o di esprimerli diversamente a seconda della fase di sviluppo software nella quale ci si trova.

Per esempio in fase di progettazione si usa generare diagrammi più dettagliati (specie nei processi organizzati a cascata), ma anche qui non importa che lo siano fino in fondo perché è veramente difficile che un progetto dettagliato sia corretto sin dal principio anche se il progettista è bravo. Quindi i diagrammi UML possono essere abbozzati e procedere poi per iterazioni successive per ampliare il diagramma ed aggiungere dettagli che rispecchiano lo stato di avanzamento della conoscenza del problema e/o dello sviluppo del sistema.

Pertanto non si può mai assumere, leggendo un diagramma che una feature non è richiesta dalla modellazione solo sulla base del fatto che non è presente in quel particolare diagramma. Sembra una manchevolezza in realtà è il punto di forza di UML e motivo della sua alta diffusione perché permette di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

essere utile anche senza riportare tutti i dettagli che talvolta o spesso possono risultare più di intralcio che di utilità. In generale troppi dettagli non servono anzi spesso sono anche controproducenti.

UML è da intendersi come uno strumento di comunicazione o di lavoro. Quindi tutti i dettagli potrebbero non essere utili o addirittura controproducenti. Anzi, andando oltre si consiglia di usare i diagrammi UML anche senza adesione stretta al formalismo standard se questo agevola nella comunicazione e le parti si intendono lo stesso.

c. Prospettive e modalità di utilizzo di UML

Un modo di differenziare l'uso di UML dipende dalla prospettiva da cui si utilizza UML per modellare, ovvero una prospettiva concettuale contrapposta ad una prospettiva software, cosa che impatta l'ambito e la modalità in cui il diagramma UML viene ad essere utilizzato.

Nella **prospettiva concettuale**, UML viene usato per la descrizione dei concetti nel dominio applicativo del problema studiato. In questo prospettiva di utilizzo viene usato per descrivere bozze di progetti o per mostrare l'idea o l'approccio di progettazione con i concetti chiave elaborati o da sviluppare. Oppure può essere usato in una **fase di analisi dei requisiti** per documentare alcuni aspetti del sistema così come sono richiesti dal committente (tramite diagrammi di casi d'uso eventualmente accompagnati da diagrammi di sequenza) o comunque in una **fase iniziale di progettazione** per rappresentare alcune parti del progetto in via di sviluppo (tramite diagramma delle classi senza dettagli su attributi o operazioni). Inoltre, in questo approccio descrittivo, non si parla più solo di elementi software, concentrandosi invece sulla costruzione di un vocabolario comune utile per discutere in un determinate dominio. Come già detto, UML può essere usato per provare a chiarire e mettere ordine sui vari significati del termine beni patrimoniali con un gruppo di contabili (tramite un diagramma delle classi).

In questo caso si parla di uso di UML come abbozzo e si tratta di solito di diagrammi informali con un basso livello di dettaglio in quanto lo scopo principale di questi diagrammi è aiutare la comunicazione e la discussione delle idee e considerare le alternative al progetto o all'approccio proposto. In effetti si possono considerare come abbozzi la maggior parte dei diagrammi UML mostrati come esempi o inclusi nelle esercitazioni, perché il loro scopo è comunicare selettivamente e non specificare in modo complete un sistema. In questo caso l'intenzione non è descrivere il progetto nel dettaglio ma solo I punti più importanti ancora in fase di definizione, quelli che si vogliono sottoporre al giudizio di colleghi o che vanno definiti prima che si cominci a programmare. In questi casi quindi lo scopo è più l'espressività e il focus su aspetti rilevanti non certo la completezza.

Nella **prospettiva software**, UML è usato per modellare i sistemi e quindi gli elementi mostrati nei diagrammi UML corrispondono (ovvero fanno riferimento) in modo diretto agli elementi di un sistema software. In questo caso si parla di uso di **UML come progetto** rivolto soprattutto alla completezza della

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

modellazione in quanto sviluppato per fornire ai programmatori un modello dettagliato da implementare. Il modello dovrebbe essere sufficientemente completo da esprimere tutte le decisioni di progetto, così che il programmatore possa seguirlo in modo abbastanza lineare, senza bisogno di interpretare alcun aspetto. Potrebbe anche darsi che il progettista e programmatore siano la stessa persona, ma in generale non lo sono (specie in grandi organizzazioni) con il primo che ha maggiore esperienza e sviluppa progetti per un'intera squadra di programmatori. Da qui la necessità di comunicare efficacemente e con chiarezza I termini con cui sviluppare le varie parti del progetto. Quest'approccio è ispirato alle altre branche di ingegneria in cui i professionisti creano progetti tecnici dettagliati che vengono poi passati alle società per la produzione.

Questo approccio descrittivo è quindi usato in **fase di progettazione** dove si punta principalmente alla completezza della modellazione e a descrivere con UML il progetto in modo dettagliato. Quando si creano progetti dettagliati si possono abbracciare tutti gli aspetti di un sistema, oppure anche concentrarsi su una particolare area. Ad esempio, un approccio comunemente usato è definire con i dettagli i modelli delle interfacce tra i vari sottosistemi, ma lasciare che siano i programmatori a sviluppare gli aspetti legati specificatamente alla loro implementazione fintantoché i dettagli specificati a livello di interfaccia sono rispettati.

Il confine tra i progetti abbozzati e quelli dettagliati non è perfettamente delineato: la distinzione risiede nel fatto che gli abbozzi sono deliberatamente incompleti e si concentrano solo sulle informazioni più importanti, mentre l'intento dei progetti è quello di essere esaustivi, spesso con lo scopo di ridurre la programmazione ad un'attività semplice e relativamente meccanica. In breve: gli abbozzi sono rivolti all'esplorazione di un problema, i progetti sono rivolti alla sua definizione e alla completezza della sua suo sviluppo ai fini della programmazione. Questo a livello di principio. Si possono avere casi di abbozzi in cui selettivamente si mostra qualche dettaglio (ad esempio nelle modalità di interazione con l'utente) oppure parti di progetto non fino in fondo specificate perché lasciate più alla decisione dei programmatori (ad esempio le parti implementative interne fintantoché le interfacce tra le classi o l'interazione con l'esterno è mantenuta come specificato).

1. Uso di UML nelle varie fasi di produzione del software

Quando si considerano i linguaggi di modellazione grafica come UML, solitamente li si inserisce nel contesto del processo a cascata che prevede un passaggio esplicito tra le fasi di analisi, progettazione e codifica, gestite generalmente da persone o gruppi di persone differenti che devono comunicare fluentemente per sapere cosa fare nello specifico ed evitare di progettare o produrre software non funzionante o non aderente ai requisiti dell'utente finale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Che si usi o no il processo a cascata, anche in altri modelli di sviluppo occorre comunque svolgere le attività di analisi, progettazione, codifica con diversi professionisti coinvolti che dovranno cooperare al meglio. Inoltre da non dimenticare la comunicazione con l'utente finale per raccogliere i requisiti e i bisogni da soddisfare.

Nell'analisi dei requisiti è importante capire la funzionalità che gli utenti si aspettano dal sistema. Inoltre nelle fasi iniziali di progettazione può essere utile abbozzare qualche elemento di sistema durante la fase di esplorazione e discussione preliminare del problema. Per far questo si può sfruttare diverse tecniche che fanno parte di UML:

- I casi d'uso descrivono l'interazione tra le persone e il sistema e quindi rappresentano i requisiti funzionali.
- Un diagramma delle classi disegnato da un punto di vista concettuale può essere di grande aiuto nella definizione di un vocabolario rigoroso da usare nella descrizione del dominio
- Un diagramma delle attività mostra il flusso di lavoro nell'organizzazione, indicando i punti di interazione tra il software e gli essere umani
- Un diagramma di attività può anche servire da contesto per i casi d'uso e chiarire il funzionamento di quelli più complessi
- Un diagramma di stato è utile se un'entità ha un ciclo di vita interessante e può assumere stati diversi che cambiano al verificarsi di determinati eventi.

In questa fase è importante usare una notazione non troppo complessa per manterenere l'utente finale coinvolto il più possibile ed evitare di introdurre info relative all'implementazione che in questa fase è inutile e potrebbe compromettere la comunicazione con l'utente.In questo caso la prospettiva di utilizzo è prevalentemente concettuale

In fase di progettazione, i diagrammi software possono diventare più tecnici, la notazione piò essere più varia e dettagliata per rispondere a scopi e modalità di lavoro differenti:

- I diagramma delle classi sono disegnati da una prospettiva di sviluppo software e quindi mostrano le classi con più dettagli, relazioni incluse
- I diagrammi di casi d'uso possono essere accompagnate da descrizioni testuali più dettagliate per gli scenari di casi d'uso che sono più significativi dal punto di vista dell'architettura o di criticità nell'interazione con l'utente.
- I diagrammi di sequenza possono documentare gli scenari più comuni o più importanti dei casi d'uso e descrivere cosa deve essere implementato nel software
- I diagrammi di package mostrano l'organizzazione del software su larga scala

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Le classi che hanno un ciclo di vista complesso possono beneficiare dei diagrammi di macchina a stato
- I diagrammi di deployment sono utili per visualizzare come dispiegare il sistema in esercizio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] A. Cockburn, Writing Effective Use Cases, 2001, Reading, M.A.: Addison-Wesley
- [2] M. Fowler, UML Distilled, Pearson, Quarta edizione
- [3] R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione
- [4] C. Larman, Applicare UML e i pattern, Pearson, Quinta edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).